

***Model-Based Systems Engineering of an
Automated Aircraft Landing Flare Maneuver***

Flare Maneuver Verification using SysML v2 and MATLAB/Simulink

AE 4699 Undergraduate Research

URA2

Academic Credit: 2 hours

By: Siddharth Balraj

Academic Course Advisor:
Prof. Dimitri Mavris

Research Advisors:
Dr. Selcuk Cimentalay
April 2026

Grade received: _____

Advisor's Signature: _____

School of Aerospace Engineering
Georgia Institute of Technology

1 Introduction

The objective of this project is to design, simulate, and formally verify a flight control system for the landing flare maneuver using a Model-Based Systems Engineering (MBSE) approach. The flare is a maneuver initiated by an aircraft approximately 50 feet (depending on the type of aircraft) above the runway during landing. During the flare, the flight control system pitches the nose up to arrest the descent rate, allowing the aircraft to touch down softly within structural and operational limits. A failure to flare correctly can result in hard landings, runway overruns, or tail strikes.

This project demonstrates the MBSE approach by constructing a complete model of the flare landing system in SysML v2, with closed-loop dynamics simulation performed in MATLAB/Simulink and results returned to SysML for automatic constraint evaluation. The SysML v2 and SysML v2 Evaluation Plugin drives the entire verification loop from architecture through analysis to formal requirement satisfaction using an integration with MATLAB.

1.1 SysML v2 over SysML v1

This project uses SysML v2, the next-generation revision of the language. Compared to SysML v1, SysML v2 offers a textual syntax that complements graphical notation, and easier support for instance specifications and analysis cases. The textual syntax also makes the model amenable to version control and external automation, both of which were valuable during the development of this project. The CATIA Magic Systems of Systems Architect 2026x environment, with the SysML v2 Plugin and SysML v2 Evaluation Plugin installed, was used as the modeling and execution platform.

2 System Requirements

The flare maneuver was decomposed into six requirements organized by category. Each was listed as a formal SysML v2 **requirement** with a constraint expression that evaluates against the simulation outputs. Table 1 lists the requirements and their rationale.

Table 1: Flare landing requirements rubric.

ID	Category	Requirement
FLT-001	Safety	Touchdown vertical velocity shall be between -3.0 and -1.0 ft/s. Sink rates more negative than -3 ft/s risk landing-gear damage; sink rates less negative than -1 ft/s indicate insufficient descent and risk runway overrun due to float.
FLT-002	Safety	Touchdown pitch angle shall be between 2.0 and 5.0 degrees. Below 2° risks nose-gear-first contact; above 5° risks tail strike.
FLT-003	Hardware	Peak elevator deflection during flare shall not exceed 12 degrees, preserving a 3° margin to the 15° actuator limit for disturbance rejection.
FLT-004	Performance	Active flare maneuver shall complete within 10 seconds to avoid runway overrun due to floating.
FLT-005	Software	The flight control computer shall not command active flare elevator deflection above the configured flare initiation altitude.
FLT-006	Comfort	Peak pitch rate during flare shall not exceed 3.0 deg/s for passenger comfort.

These requirements are defined under a parent requirement **LandingPerformance**, which an instance must satisfy via a `satisfy landingRubric` usage. Each individual constraint is evaluated independently and rolls up into the overall pass/fail of the parent (each requirement must pass for the landingRubric to pass). Figure 1 shows the **LandingPerformance** requirement definition as captured in the SysML v2 model, with each of the six sub-requirements visible alongside their constraint expressions.

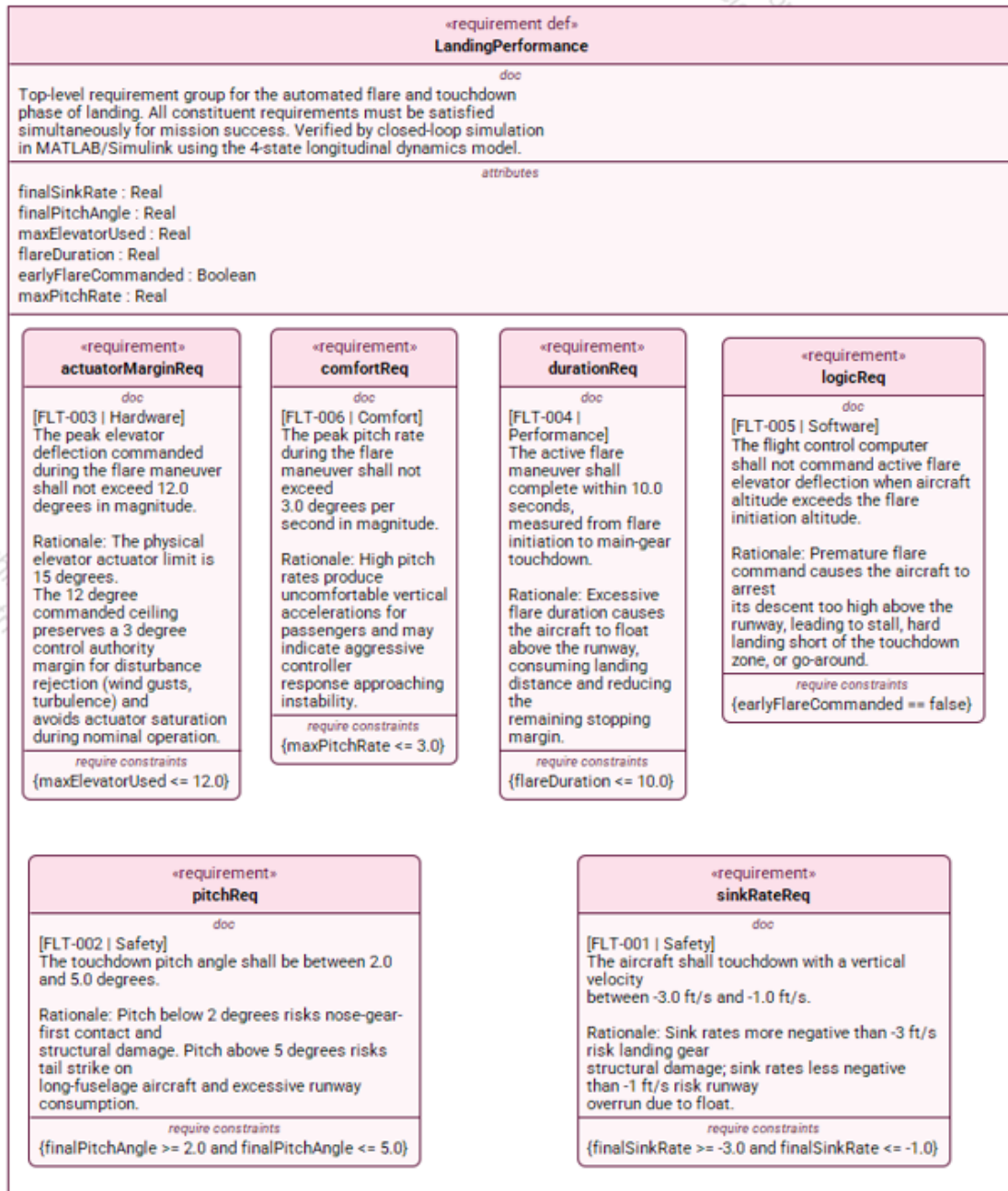


Figure 1: LandingPerformance requirement definition in the SysML v2 model.

3 System Structure and Behavior

3.1 Architecture

The aircraft system is modeled as a top-level part, **AircraftSystem**, composed of four parts that mirror the logical decomposition of a flare control loop:

- **RadarAltimeter** measures altitude above the runway.

- **FlightControlComputer** hosts the flare controller logic and commands the actuator. It carries the controller configuration parameters: the desired flare altitude and the target sink rate of the aircraft after the flare.
- **ElevatorActuator** converts a control command into a physical elevator deflection, bounded by a maximum elevator deflection parameter.
- **AirframeDynamics** carries the physical parameters of the airframe (mass, wing area, air density, approach speed, initial sink rate).

Figure 2 shows the system structure diagram for the **AircraftSystem** part. The connections between parts and the flow of data can be observed by the ports and the arrow directions between the parts.

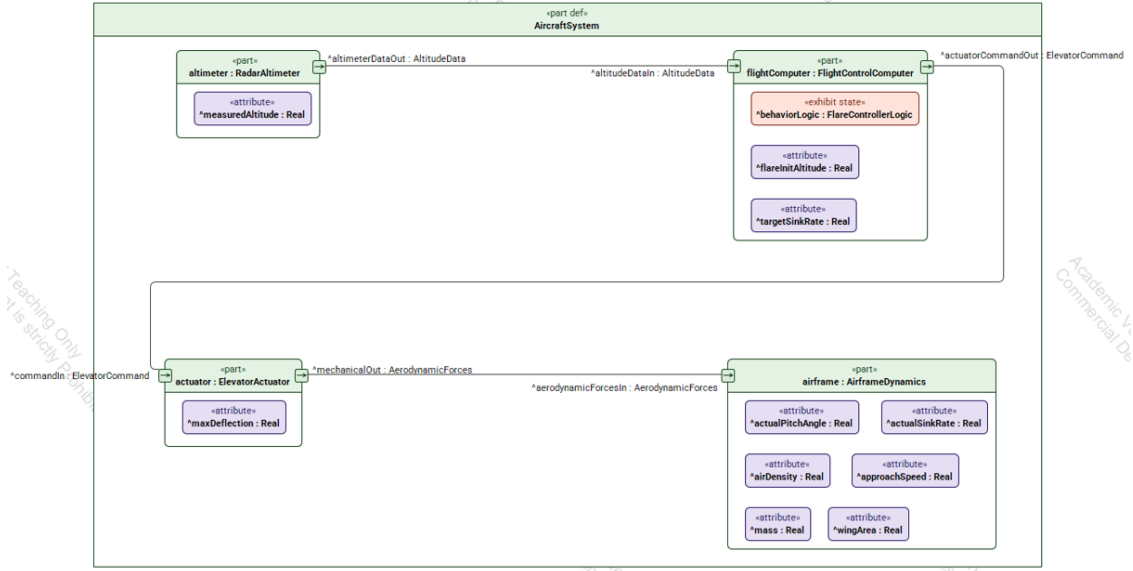


Figure 2: System structure diagram of the AircraftSystem.

Connections between parts model the data flow through the loop: altimeter → flight control computer → actuator → airframe. This forms a closed feedback loop where the airframe's altitude is measured and fed back to the controller through the altimeter.

3.2 Behavior

The flare controller logic is captured as a SysML v2 state definition, **FlareControllerLogic**. It defines four states linked by guarded transitions:

Initial → **ApproachTracking** → **FlareMode** → **TouchdownRoll** → **Final**

Figure 3 shows the system behavior diagram which highlights the flow through each state. The **ApproachTracking** state represents stabilized glideslope tracking with no active flare command. When altitude drops below the desired flare altitude, the machine transitions to **FlareMode**, where the LQI controller actively tracks a commanded sink rate. When altitude reaches zero, the machine transitions to **TouchdownRoll**, marking ground contact. The transition guards reference the **flareInitAltitude** (desired flare altitude) attribute on the **FlightControlComputer** instance, which shows the parameter traceability from instance specification to behavioral logic.

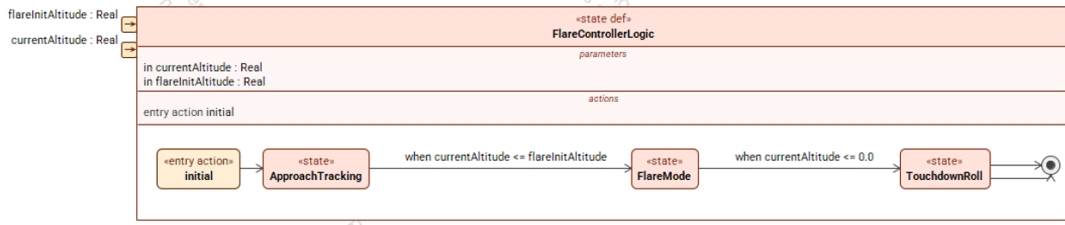


Figure 3: System behavior diagram of the FlareControllerLogic.

4 Dynamics Modeling

The closed-loop simulation requires a mathematical model of the aircraft's longitudinal flight dynamics during the final approach and flare. The full nonlinear rigid-body equations were linearized about a steady-approach trim condition to produce a 4-state state-space representation suitable for optimal control synthesis.

4.1 State Vector and Linearization

The state vector is $X = [u, w, q, \theta]^T$, where u and w are the body-frame forward and downward velocity perturbations, q is pitch rate, and θ is pitch angle perturbation. The control input is the elevator deflection δ_e . Altitude is integrated separately from the state-space dynamics because it serves only as a triggering signal (when flare should be activated) and termination condition (when altitude equals zero).

The trim pitch angle is computed as $\theta_0 = \alpha_0 + \gamma_0$, where α_0 is the approach angle of attack (nominally 3°) and $\gamma_0 = \dot{h}_0/u_0$ is the flight path angle inferred from the initial sink rate.

4.2 State-Space Model

After Taylor expansion of the aerodynamic forces and moments about trim and applying the small-disturbance approximation, the linearized model takes the form $\dot{X} = AX + BU$, $Y = CX$, with:

$$A = \begin{bmatrix} X_u & X_w & 0 & -g \cos \theta_0 \\ Z_u & Z_w & u_0 & -g \sin \theta_0 \\ 0 & M_w & M_q & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ M_{\delta_e} \\ 0 \end{bmatrix}, \quad C = \begin{bmatrix} 0 & -1 & 0 & u_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The output matrix C extracts sink rate ($\dot{h} = u_0\theta - w$ under small angle) and pitch angle from the state. The dimensional stability derivatives ($X_u, Z_w, M_q, M_{\delta_e}$, etc.) are populated directly from physical parameters supplied by the SysML instance: aircraft mass, wing area, air density, and approach speed. Geometric quantities (mean aerodynamic chord, moment of inertia I_{yy}) are estimated from the instance provided wing area using a transport-aircraft aspect ratio of 9 and an empirical radius-of-gyration coefficient.

5 LQI Controller Design

A Linear Quadratic Integral (LQI) controller was chosen to track a commanded sink rate to touchdown, with integral action eliminating steady-state tracking error.

The system has a single control input, the elevator, which means only one output can be independently commanded at a time. Sink rate is chosen as that output because it is the primary safety-critical parameter at touchdown. Pitch angle is left uncontrolled — when the elevator deflects upward to reduce sink rate, the aircraft naturally pitches up as part of the flare, and with correct tuning this response falls within the 2° to 5° safety window without needing to be commanded directly.

The controller gain matrix was computed by using the LQI function in MATLAB with the following state and control effort weights:

$$Q = \text{diag}(0.001, 0.001, 1.0, 0.0, 200), \quad R = 500$$

The weight on the pitch angle state is set to zero so the controller does not resist the pitch-up motion during the flare. The weight on the integral state (200) keeps sink rate tracking tight. The control effort weight $R = 500$ keeps the elevator motion smooth and prevents saturation against the 15° actuator limit.

5.1 Altitude-Scheduled Reference

Commanding the target sink rate of -2 ft/s as an immediate step at the flare initiation altitude is not physically achievable since the elevator cannot arrest 10 ft/s of descent instantaneously without saturating. Instead, the commanded sink rate is ramped linearly with altitude:

$$r(h) = \begin{cases} \dot{h}_0 & h > h_{\text{flare}} \\ \dot{h}_{\text{target}} + \frac{h}{h_{\text{flare}}} (\dot{h}_0 - \dot{h}_{\text{target}}) & 0 \leq h \leq h_{\text{flare}} \end{cases}$$

At the flare initiation altitude, the command matches the current sink rate exactly, so there is no sudden jump. As the aircraft descends, the command smoothly reduces toward the touchdown target. This allows the controller to operate without saturating the elevator throughout the maneuver.

6 Simulink Implementation

The Simulink block diagram implements the closed-loop flare simulation using the state-space plant and LQI controller established in MATLAB. The plant outputs sink rate and pitch angle, which are compared against the altitude-scheduled reference to generate the tracking error. The error is gated by an altitude switch that activates the controller only below the flare initiation altitude, preventing early elevator commands. A saturation of the actuator limit is applied to the controller output before feeding back into the plant. A separate altitude integrator drives the simulation stop condition at ground contact ($h=0$).

The model logs six output signals — sink rate, pitch angle, pitch rate, elevator deflection, altitude, and time — which are post-processed in MATLAB to produce the six rubric metrics returned to SysML. Figure 4 shows the Simulink block diagram modeled to run the flare simulation, with the state-space plant, controller gain, reference scheduling, and signal logging blocks visible.

8 Instance Verification

Two aircraft instances were modeled and verified against the rubric: a heavy commercial jet (mass 350,000 lb, wing area 3050 ft², approach speed 250 ft/s) and a small general-aviation aircraft (mass 2500 lb, wing area 174 ft², approach speed 110 ft/s). The same MATLAB function and simulation are performed for both types of aircraft, just with different parameters based on the input. Figures 5 and 6 show the evaluated SysML instances for the heavy commercial jet and the smaller aircraft respectively. Each figure displays the parts and their attribute values, the simulation output values, and the pass/fail status of each requirement.

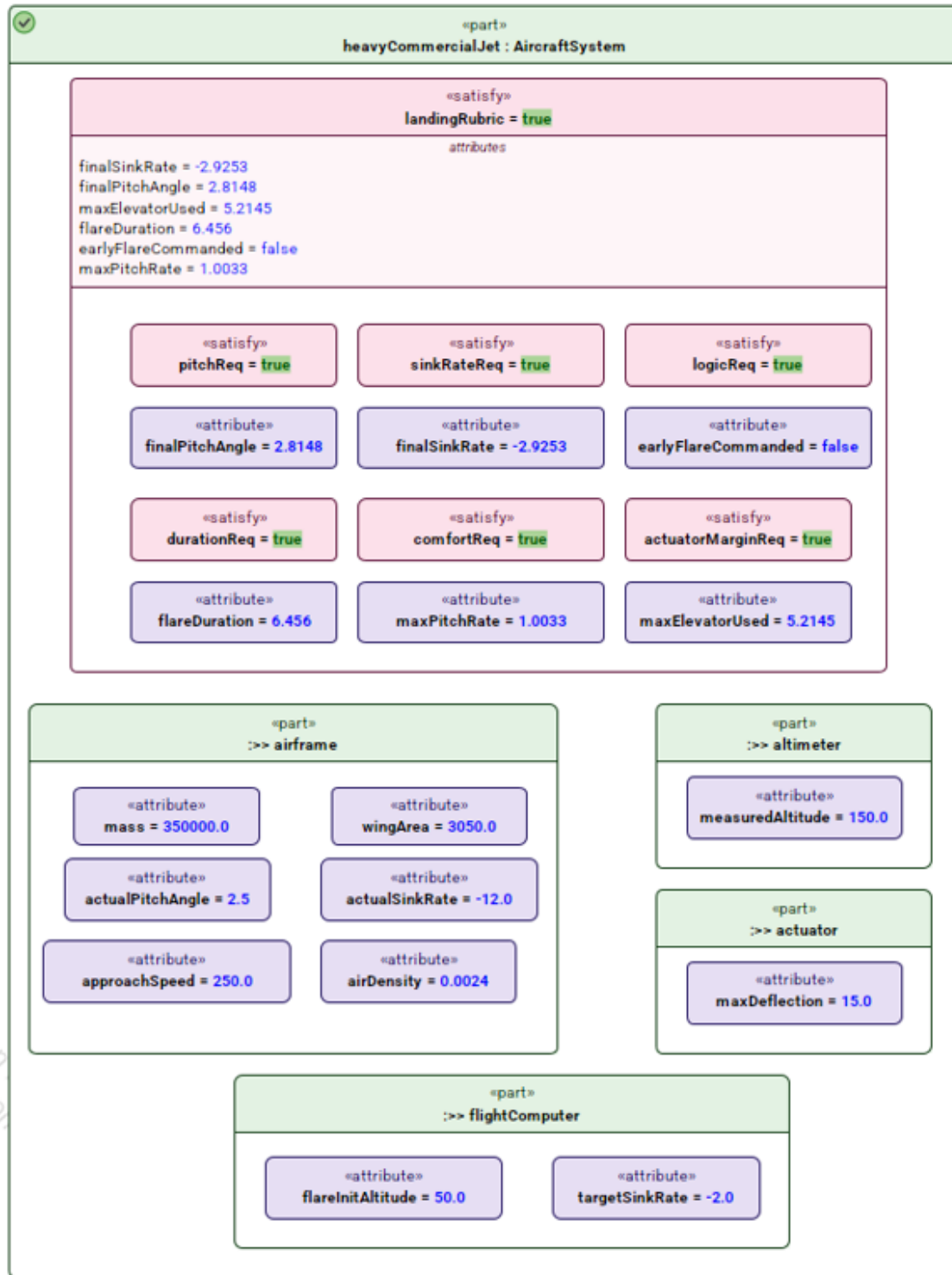


Figure 5: Evaluated SysML instance for the heavy commercial jet aircraft.

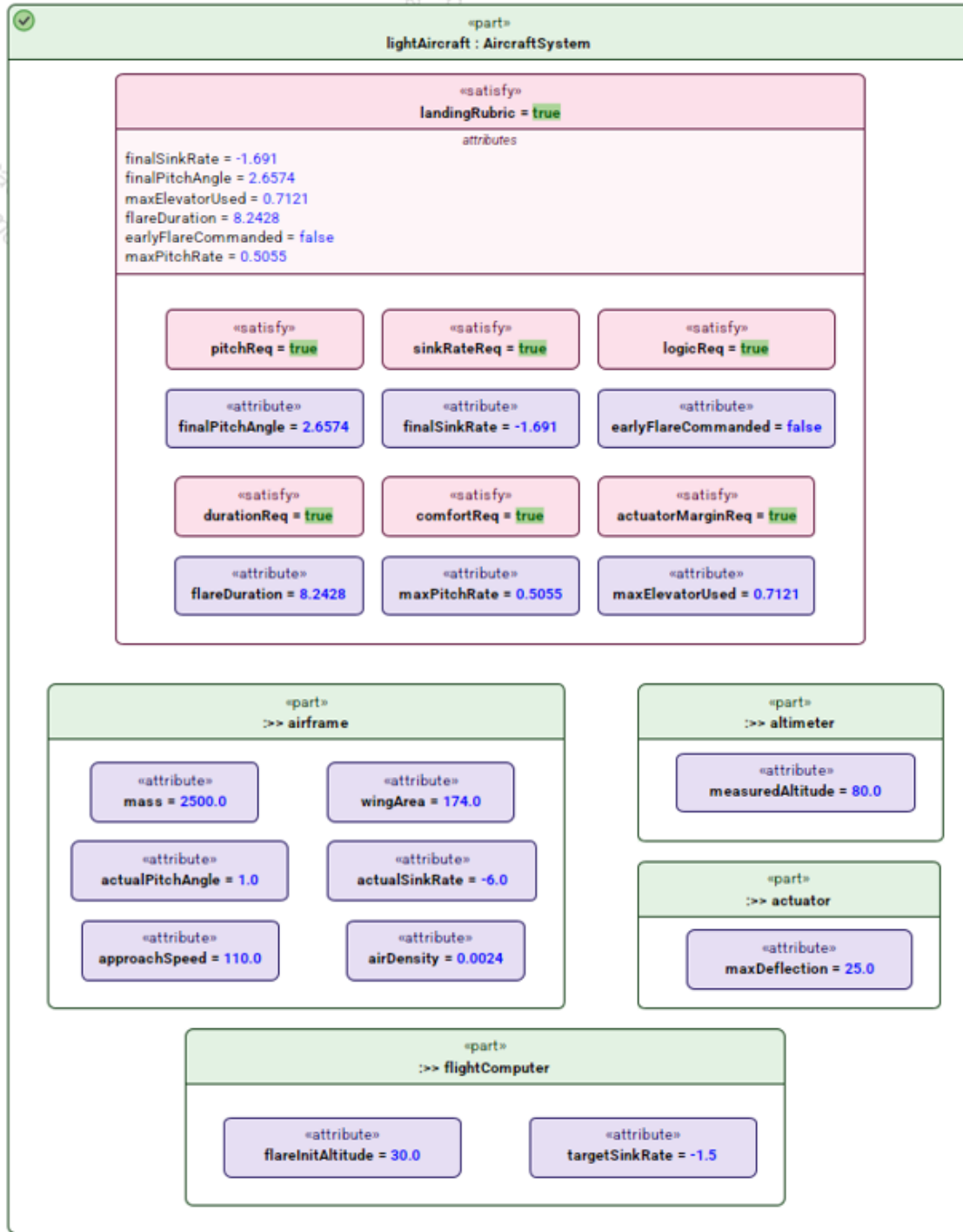


Figure 6: Evaluated SysML instance for the smaller general-aviation aircraft.

Tables 3 and 4 show the verification results for the heavy jet and the smaller aircraft against each requirement.

Table 3: Heavy commercial jet verification results.

Requirement	Computed Value	Limit	Status
FLT-001: Sink rate (ft/s)	-2.93	$[-3, -1]$	Pass
FLT-002: Pitch angle (deg)	2.81	$[2, 5]$	Pass
FLT-003: Max elevator (deg)	5.21	≤ 12	Pass
FLT-004: Flare duration (s)	6.46	≤ 10	Pass
FLT-005: Early flare (boolean)	false	false	Pass
FLT-006: Max pitch rate (deg/s)	1.00	≤ 3	Pass

Table 4: Smaller general-aviation aircraft verification results.

Requirement	Computed Value	Limit	Status
FLT-001: Sink rate (ft/s)	-1.69	$[-3, -1]$	Pass
FLT-002: Pitch angle (deg)	2.66	$[2, 5]$	Pass
FLT-003: Max elevator (deg)	0.72	≤ 12	Pass
FLT-004: Flare duration (s)	8.24	≤ 10	Pass
FLT-005: Early flare (boolean)	false	false	Pass
FLT-006: Max pitch rate (deg/s)	0.51	≤ 3	Pass

All six requirements pass with comfortable margin for the transport jet. The same evaluation against the light aircraft instance also passes, demonstrating that the system architecture is robust across a wide range of airframes when its parameters are correctly populated from the instance specification.

8.1 Demonstration of Requirement Failure

To verify that the rubric correctly evaluated realistic maneuver cases, the heavy jet’s mass was increased from 350,000 lb to 500,000 lb (an off-nominal overweight condition) and the model re-evaluated. The increased mass causes the controller to be unable to fully arrest the descent rate within the available altitude. The resulting touchdown sink rate of -3.35 ft/s exceeds the -3.0 ft/s safety floor, and FLT-001 fails. The remaining five requirements still pass, and the overall `landingRubric` results as false. This demonstrates that the verification rubric correctly identifies design conditions outside the safe operating envelope. Figure 7 shows the evaluated SysML instance for this overweight failure case, with FLT-001 marked as failed and the parent `landingRubric` aggregating to false.

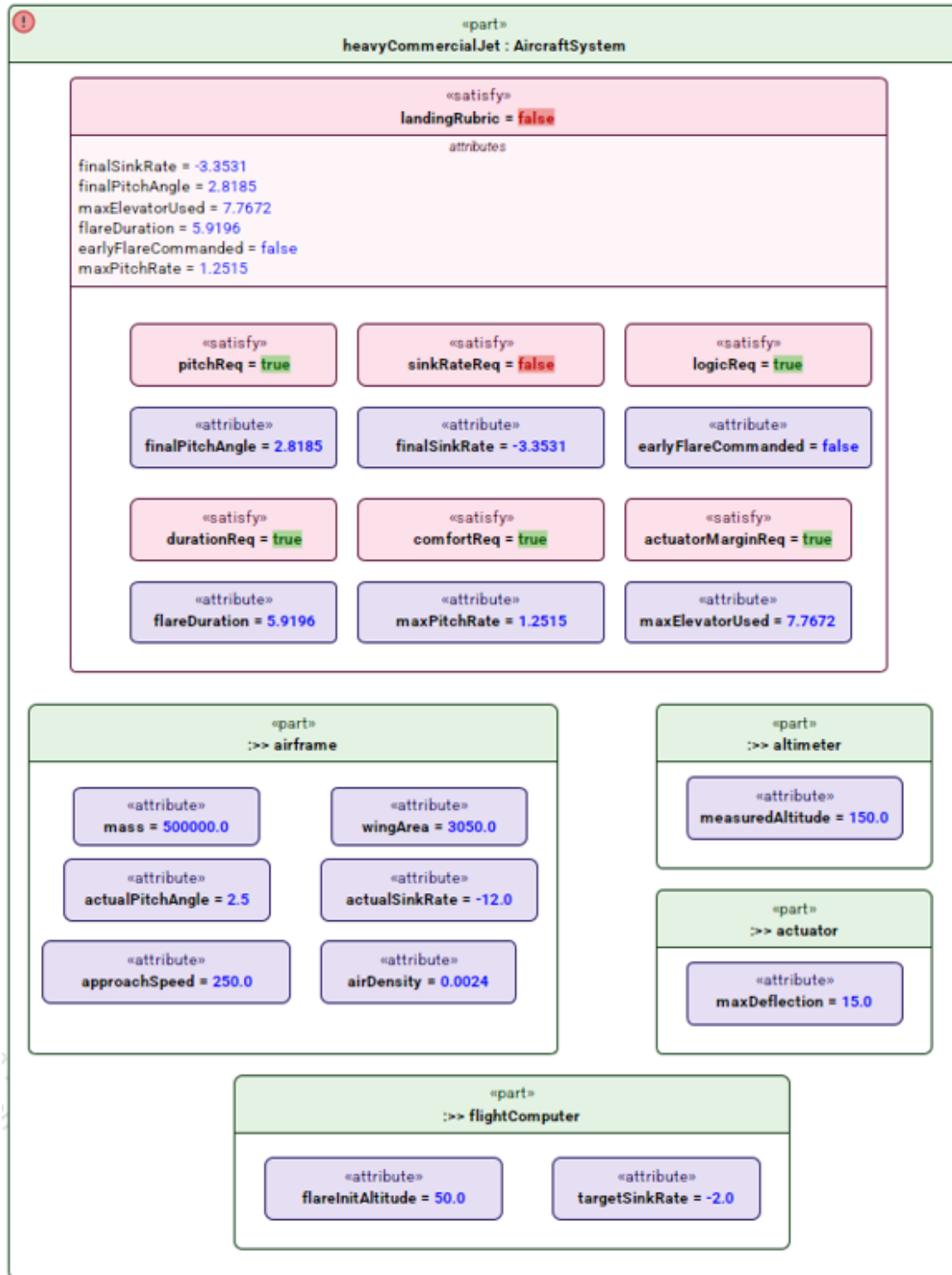


Figure 7: Evaluated SysML instance for the overweight failure case (mass = 500,000 lb).

9 Conclusions

This project successfully implemented and verified an automated landing flare control system using a complete MBSE workflow in SysML v2. The system architecture, behavior, requirements, instance specifications, and analysis are captured in a single SysML model. MATLAB and Simulink provide the dynamics simulation and controller synthesis, and the SysML v2 Evaluation Plugin closes the loop by executing the simulation and evaluating the resulting attribute values against the requirement constraints.

The verification results confirm that the simulated control system satisfies all six landing requirements for both modeled aircraft instances. An off-nominal test (aircraft with overweight mass) correctly produced a requirement failure. The project demonstrates how MBSE tooling can integrate architecture, simulation, and verification into a traceable model.

Future work could extend the model to include multiple degrees of freedom dynamics for cross-wind landings, add disturbance models (gust, turbulence), and replace the linear plant with a higher-fidelity airframe model to evaluate and validate more complex conditions.